Sentinela — Roteiro de apresentação

João Maurício Araújo Leitão
13/11/2026

1. ABERTURA (30 S)

> *"O Sentinela recebe telemetria veicular via OBD-II, padroniza e armazena em PostgreSQL e usa **três métodos** — limites do CTB, z-score histórico e Machine Learning — para detectar e classificar eventos de risco, gerando um **score de segurança por quilômetro** e relatórios visuais."*

Transição: "Antes do código, o problema: como saber se uma viagem foi perigosa?"

2. ANTES DE APRESENTAR (5 MIN)

```
1) Abrir o Docker Desktop ("Running")
cd C:\Users\jo22m\Documents\Trabalho_PAV\projeto_api
docker compose up -d && docker compose ps

2) powershell -c "Invoke-RestMethod
http://localhost:8080/telemetria/rotas"

3) cd ..\projeto_interface && python main.py
```

- **Backup:** PDF dos slides + 1 relatório HTML já gerado.
- NUNCA rodar `docker compose down -v` (apaga o banco).
- CSVs da demo em `apresentacao\dados_teste\`.

3. DEMO (7 MIN)

#	ONDE	MOSTRAR / DIZER
1	Terminal	`ps` + `GET /rotas`: a GUI fala HTTP, não com o banco
2	Dados → Rotas	Nova Rota → `Conducao_Agressiva.csv`
3	Visão Geral	Score 0–100 normalizado por km + KPIs + eventos por tipo
4	Gráfico	Zoom + Painel (Velocidade / Força G / RPM); pontos = eventos
5	Sensores	Marcar **"Todas as rotas"** → agregado
6	Comparar	Agressiva × Defensiva
7	Relatório	Gerar HTML e abrir no navegador
8	Dados → Eventos	Método: LIMITE / ESTATISTICO / MODELO_ML
9	Slides	Arquitetura · slide 8 (classes) · slide 9 (z-score)

Transição: "Agora que vocês viram o resultado, vou mostrar como o sistema decide."

4. ENDPOINTS (13 CAMINHOS)

Base: `http://localhost:8080/telemetria`

MÉTODO	ENDPOINT	FUNÇÃO
GET/POST	`/rotas`	Listar / cadastrar
POST	`/rotas/importar`	Importar CSV
GET/PUT/DEL	`/rotas/<id>`	Detalhe / editar / excluir
POST	`/rotas/<id>/analisar`	Reanálise forçada
GET	`/rotas/<id>/resumo`	Score + KPIs
GET	`/rotas/<id>/grafico`	Pontos do gráfico
POST	`/rotas/comparar`	2 a 5 rotas
GET/POST	`/leituras?rota_id=`	Listar / cadastrar
GET/PUT/DEL	`/leituras/<id>`	Detalhe / editar / excluir
GET/POST	`/eventos-risco?rota_id=&tipo=`	Listar / cadastrar
GET/PUT/DEL	`/eventos-risco/<id>`	Detalhe / editar / excluir
GET	`/limiares`	Somente leitura
GET	`/sensores`	Catálogo S01…S08

Status: 200 ok · 201 criado · 400 validação · 404 não encontrado · 409 duplicado · 405 proibido · 500 interno.

5. PERGUNTAS PROVÁVEIS

PERGUNTA	RESPOSTA
Onde está a regra X?	Tabela do README mapeia as 10 regras
Por que 3 tabelas?	`rota → leitura → evento_risco`; limiares/sensores em JSON
Por que banco vazio?	Requisito; `seed.py` é opcional
Como o ML decide?	StandardScaler → DBSCAN(eps 1,0; min 15) → RandomForest
Com quais dados treina?	Com as **outras** rotas; fallback para a própria
Por que z-score 3?	Regra 3σ (~99,7%); configurável
Como o score funciona?	`100 − 2,0 × penalidade/km`, pesos por tipo
Como evita duplicata?	`UNIQUE (rota_id, data_hora)` e `(leitura_id, metodo, classificacao, canal)`
Onde há herança?	`DetectorAnomalia` → 3 detectores

6. PONTOS FRACOS (ADMITA COM MATURIDADE)

- **OBDLink com score 0:** ruído no acelerador (634 eventos) → melhoria: suavização.
- `forca_g_max` **no gráfico mas não no método LIMITE:** os métodos 2 e 3 capturam; melhoria: incluir no `LimiteDetector`.
- **Agregação por leitura:** rotas longas pesam mais (decisão consciente).
- **ML com pseudo-rótulos:** sem ground truth; `avaliar_ml.py` mede silhouette.

Transição: "Por fim, o que ficou como melhoria."

7. PLANO B

SE FALHAR...	FAÇA
Docker não subir	Relatório HTML + slides + capturas em `apresentacao/imgs`
API cair	`docker compose restart`; a GUI reconecta sozinha
Import falhar	Usar as rotas já no banco
PowerPoint não abrir	PDF de backup

8. NÚMEROS E EXTRAS

3 tabelas · **3** métodos · **7** classificações · **10** regras · **13** endpoints · **19** slides · **6** rotas · **18.984** leituras

Se sobrar tempo: `test_api.py` (testes verdes) · `reanalisar.py` (novos limiares) · `avaliar_ml.py` (métricas do modelo).